2.3 Unidentifiability

The unidentifiability of tasks forms a key part of our framework. It directly addresses the fundamental challenge that models, such as LLMs, face when dealing with unseen tasks. If a task λ lies outside of $\text{Span}(\Phi)$, the span of tasks the model has been trained on, then the model cannot be expected to infer or apply it correctly. In other words, the model's capacity is constrained by the identifiability of tasks within the training set. This notion and formalization of unidentifiability allows us to highlight a critical limitation in the generalization of models: tasks not encountered during training cannot be reliably executed, as they remain beyond the model's learned task-span. More formally:

Definition 2.4 (Unidentifiability)**.** A task λ is unidentifiable in a set $\Phi \subseteq \Upsilon$ if and only if $\lambda \notin \text{Span}(\Phi)$.

Remark In practice, unidentifiability may depend on the initial state X_0, i.e. an LLM might accurately perform addition for 2-digit numbers but fail with 10-digit numbers [24]. For more details, see Appendix B.1.

Building on this framework, we propose an algorithm that integrates unidentifiability with information-theoretic methods to detect CoT reasoning failures.

3 Operationalising Our Framework

This section aims to operationalise the above framework to make inferences regarding the unidentifiability of intermediate sub-tasks in a model's CoT reasoning process. This would subsequently allow us to detect any sub-task at which a model's CoT reasoning process starts to diverge from the ground truth, thereby providing insights into how the model can be improved. For example, suppose we are in a setting where the "addition" operation is unidentifiable, then we could further improve the model's mathematical reasoning by fine-tuning it on the addition operation.

3.1 An information-theoretic perspective

To apply unidentifiability in CoT generations, we introduce a fundamental assumption: each correct CoT step should provide meaningful information aiding the prediction of Y. If a step ceases to increase information about Y, it indicates an incorrect execution. We formalize this assumption using our notation from the previous section:

Assumption 3.1 (Bayesian network)**.** Let $\lambda \neq \lambda'$ be two operations with primitive decompositions:

$$\lambda = \lambda_1 \circ \ldots \lambda_{k-1} \circ \lambda_k \circ \cdots \circ \lambda_T \quad \text{and}$$
$$\lambda' = \lambda_1 \circ \ldots \lambda_{k-1} \circ \lambda'_k \circ \cdots \circ \lambda'_{T'},$$

where λ'_k is unidentifiable in $\{\lambda_1, \ldots, \lambda_T\}$. Then, the intermediate states corresponding to the tasks λ, λ' have the Bayesian network in Figure 2.

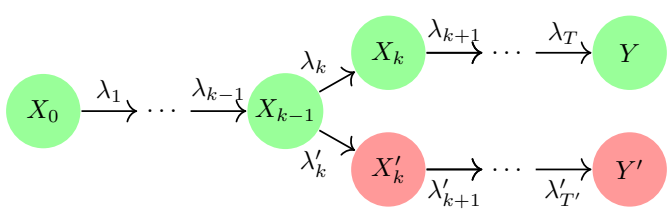

Figure 2 Bayesian network corresponding to Assumption 3.1.

Intuition The Bayesian network in Figure 2 implies that if we encounter an unidentifiable task (λ'_k) at step k of the reasoning path, the future states X_i and X'_j for any $i, j \geq k$ satisfy the conditional independence $X_i \perp\!\!\!\perp X'_j \mid X_{k-1}$. Consequently, once we apply λ'_k, the subsequent states along the new reasoning path (in red) add no information regarding the subsequent states or the output of the original path (in green). Hence

the figure represents the fact that, for any given input, the output of λ_k (top fork) contains no information regarding the output of any other primitive task λ'_k (bottom fork).

With our key assumption on the ground-truth CoT process formalized, we now consider the model's CoT behaviour.

3.2 Task execution in LLMs

To operationalise our framework, we formally distinguish between the model i.e. LLM's task execution and the ground truth process which arises from following the instructions correctly. To this end, we explicitly define how an LLM interprets a specified task λ using $\Lambda^M(X_0, \lambda)$, which is in general distinct from the ground truth update rule $\Lambda(X_0, \lambda)$.

Here, one option would be to consider the idealised setting where the model learns to perfectly follow some of the primitive tasks available in the training data. However, this may be considered too restrictive since in reality most LLMs do not always follow a "learned" task perfectly. Instead, we consider a much weaker assumption that the model cannot correctly execute a task which is unidentifiable in the training data. Concretely, suppose $\Gamma^M \subseteq \Gamma$ denotes the primitive tasks available in the LLM training data. Then, we make the following assumption on LLM's task execution.

Assumption 3.2 (Task execution in LLMs)**.** Λ^M is compositionally consistent and for any $(x_0, \lambda) \in \mathcal{X} \times \Upsilon$, there exists some $\widehat{\lambda} \in \text{Span}(\Gamma^M)$ such that $\Lambda^M(x_0, \lambda) \stackrel{\text{d}}{=} \Lambda(x_0, \widehat{\lambda})$.

Intuition Assumption 3.2 means that for any task which we would like the LLM to apply, the LLM ends up executing some task in $\text{Span}(\Gamma^M)$ which the model has been trained on. In other words, the model's execution is restricted only to the tasks which could be inferred from the training data (i.e. in $\text{Span}(\Gamma^M)$). Moreover, this assumption also allows us to encapsulate cases where the model does not follow correct instructions or does not decompose a task correctly.

Before proceeding further with our main result which will allow us to test for the unidentifiability of sub-tasks, we define some notation which we will use from now onwards. Let $\lambda = \lambda_1 \circ \ldots \circ \lambda_T$ denote a primitive decomposition of a task λ. Then, starting from an initial state X_0, we denote the model's intermediate states recursively as:

$$X_t^M := \Lambda^M(X_{t-1}^M, \lambda_t) \quad \text{and} \quad X_0^M = X_0.$$

Moreover, we use Y^M to denote the model's final output. Next, using this notation, we present the conditional independence which must hold if the model encounters an unidentifiable intermediate task along its reasoning path.

Theorem 3.3. *Let $\Gamma^M \subseteq \Gamma$ denote the primitive tasks available in the training data. Let λ be a task with decomposition $\lambda = \lambda_1 \circ \ldots \circ \lambda_T$. If λ_k is the first task in the decomposition of λ which is unidentifiable in Γ^M (i.e. $k = \arg\min_t\{\lambda_t \notin \text{Span}(\Gamma^M)\}$). Then, under Assumptions 3.1 and 3.2, we have that*

$$Y \perp\!\!\!\perp X_j^M \mid X_{j-1}^M \quad \text{for all } j \geq k. \tag{2}$$

Theorem 3.3 shows that under Assumptions 3.1 and 3.2, when the model encounters an unidentifiable task (i.e. λ_k in Theorem 3.3) in its Chain-of-Thought reasoning, the model output satisfies the conditional independence in Equation (2). In practice, this means that if at step k, a model encounters a reasoning step which is necessary for obtaining the correct answer and is unidentifiable in the training data, the CoT reasoning diverges from the ground truth at this step and every subsequent step adds no additional information regarding the correct final output Y. This 'information' can be measured by checking if the model's confidence about the final output Y increases after each step. This is formalised in the next section.

3.3 Testing for unidentifiability using information-gain

With our framework established, we now describe how to detect unidentifiable sub-tasks using information theory. Following common practice [11, 29], we assume access to a dataset of prompts and correct final

answers, derived by applying task λ, denoted as $\mathcal{D}_\lambda := \{(x_0^i, y^i)\}_{i=1}^n$. Recall that X_j^M and X_{j-1}^M represent the model's chain of thought (CoT) reasoning at steps j and $j-1$, respectively. Consequently, each element in the conditional independence statement in Equation (2) can be derived from the data and/or the model.

To this end, we consider the mutual information between Y and X_j^M conditional on X_{j-1}^M, denoted by $\mathcal{I}(Y; X_j^M \mid X_{j-1}^M)$. This conditional mutual information term intuitively represents the additional information contributed by the j'th step of CoT, which is relevant for predicting the ground truth final output Y. Therefore, we refer to $\mathcal{I}(Y; X_j^M \mid X_{j-1}^M)$ as the information-gain at step j.

It follows from Theorem 3.3 that if an LLM encounters a sub-task at step i which is unidentifiable in its training data, no subsequent step should contribute any additional information relevant for predicting Y (i.e. the information-gain should remain 0 after step i). If, on the other hand, we observe that $\mathcal{I}\left(Y; X_j^M \mid X_{j-1}^M\right) > 0$ for some $j \geq i$, then under Assumptions 3.1 and 3.2, the task λ_i is not unidentifiable. To estimate the information-gain in practice, we use the following result:

Proposition 3.4. *Let $\mathcal{I}(X; Y \mid Z)$ denote the mutual information between X and Y conditional on Z. Then,*

$$\mathbb{E}[\log p(Y \mid X_j^M)] - \mathbb{E}[\log p(Y \mid X_{j-1}^M)] = \mathcal{I}\left(Y; X_j^M \mid X_{j-1}^M\right) \geq 0. \tag{3}$$

To estimate the information-gain in (3) using Proposition 3.4, we train a separate LLM, which we refer to as the supervisor model g_{sup}. This model takes as input the model's CoT reasoning up to any given intermediate step t, X_t^M, and is fine-tuned to directly predict the ground truth final output Y. In this way $g_{\text{sup}}(X_t^M)$ approximates the conditional distribution $p(Y \mid X_t^M)$. Then, the quantity $\mathbb{E}[\log p(Y \mid X_j^M)]$ can be estimated using the negative cross-entropy loss for predicting Y, i.e., $\mathbb{E}[\log p(Y \mid X_j^M)]$ is approximately

$$\mathbb{E}[\log \hat{p}(Y \mid X_j^M)] = -\mathbb{E}[l_{\text{CE}}\left(Y, g_{\text{sup}}(X_j^M)\right)],$$

where l_{CE} denotes the cross-entropy loss. From this, it follows that

$$\underbrace{\mathbb{E}[\log p(Y \mid X_j^M)] - \mathbb{E}[\log p(Y \mid X_{j-1}^M)]}_{\text{Information-gain}} \approx \mathbb{E}[l_{\text{CE}}(Y, g_{\text{sup}}(X_{j-1}^M))] - \mathbb{E}[l_{\text{CE}}(Y, g_{\text{sup}}(X_j^M))]. \tag{4}$$

Summary The information-gain (IG) between steps j and $j-1$ reflects how much relevant information step j contributes towards predicting Y. If task λ_j is executed correctly, this gain is positive, as indicated by a decrease in the cross-entropy loss. Conversely, if step j does not provide additional information, the loss remains unchanged. This can be interpreted as the conditional mutual information between X_j^M and Y, conditioned on X_{j-1}^M. Positive information-gain suggests step j adds new insight about Y, while no gain indicates no added information. Training details for the supervisor model are in Appendix C.1.3.

Remark on sample-wise information-gain While conditional mutual information provides an aggregate measure of information-gain for a sub-task in a dataset, it may also be desirable to obtain an analogous measure of sub-task correctness for individual CoT instances. This could be useful, for example, in detecting which step is wrong for a given prompt. Our notion of information-gain can be extended to this sample-wise setting, similar to [8], by instead considering the following difference

$$\log p(Y \mid X_j^M) - \log p(Y \mid X_{j-1}^M) \approx l_{\text{CE}}(Y, g_{\text{sup}}(X_{j-1}^M)) - l_{\text{CE}}(Y, g_{\text{sup}}(X_j^M)). \tag{5}$$

Intuitively, if step j in the model's CoT is correct, the model should become more confident in the ground truth output Y being the correct final answer. Therefore, the difference above should be positive. Conversely, if step j is wrong, the model's confidence regarding Y should not increase, and this difference should be ≤ 0. From now on, we refer to the difference in (5) as sample-wise information-gain at step j.

Remark on O1/R1 style reasoning Although we present our framework using linear chains-of-thought for clarity, the information-gain metric naturally accommodates the more complex reasoning patterns found in O1/R1-style models. These reasoning models often explore multiple solution paths, backtrack when encountering errors, and dynamically self-correct their approach. In such exploratory settings, our framework remains effective: steps along incorrect reasoning trajectories will exhibit low or negative information gain,